

Overview

`GraphBuilder` now ensures every node has a unique identifier.

A new helper `uniqueNodeId` (lines 84-109) builds deterministic IDs that optionally include a line number and, if needed, a numeric suffix to avoid collisions.

Node-creation methods have been updated to use this helper, and file nodes are now added only if the ID is not already present (lines 146-157).

Key changes

- `uniqueNodeId` – generates collision-free IDs (lines 84-109).
- `addFileWithAnalysis` – checks `this.nodeIds.has(fileId)` before adding a file node (lines 146-157).
- **Function & class nodes** – created with `this.uniqueNodeId(..., fn.lineRange[0])` and `cls.lineRange[0]` (lines 161-167, 191-197) instead of hard-coded IDs.
- `addChildNode` – now uses `uniqueNodeId` to dedupe child nodes and logs a warning on duplicates (lines 344-351).
- `addNonCodeFileWithAnalysis` – delegates all child creation to the updated `addChildNode`, ensuring consistent ID handling for definitions, services, endpoints, steps, and resources.

Impact

- **Correctness** – eliminates accidental duplicate nodes that could corrupt graph queries.
- **Maintainability** – central ID logic reduces duplication and future bugs.
- **Observability** – console warnings surface duplicate-ID attempts, aiding debugging.

Risks & follow-ups

- **Regression** – tests that assert specific node IDs may fail; run the full graph test suite to confirm.
- **Edge cases** – verify that files with identical names and line ranges across projects still produce distinct IDs via the suffix logic.
- **Logging noise** – console warnings could clutter production logs; consider a configurable logger if needed.
- **Compatibility** – no public API changes, so downstream consumers remain unaffected.